PERCONA

Databases run better with Percona

PERCONA

About Me

JOBIN AUGUSTINE (48)

- Working for Percona
- Life with databases

PERCONA

PostgreSQL Indexing

Common Pitfalls and Misconceptions

JOBIN AUGUSTINE

Why This topic is important - human factor

- Microscopic view
- Looses larger picture
- Unaware about the consequences

- Typically, the focus is on the positive aspects, with little attention given to potential negative impacts.
- Indexes are a common area where this oversight occurs, often leading to significant damage.
- Maintaining a 360-degree awareness is crucial.

©2024 Percona

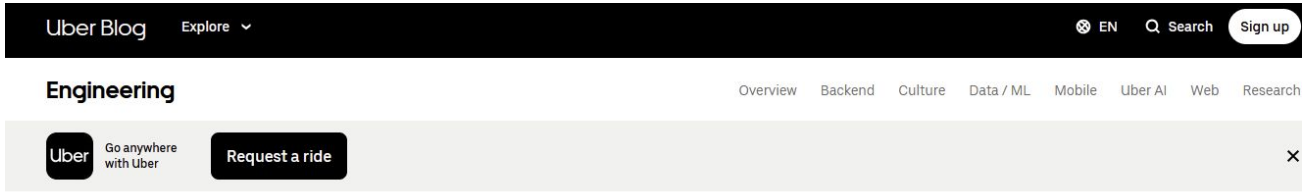
1. Indexes penalize the transactions

- Indexes could speed up data retrieval (SELECT)
- But, at the cost of transaction processing (DMLs)
  - It can jeopardize the TPS objectives
- Each transaction need to update indexes

- Create a mental picture of what could be happening
- Avoid falling prey to testimonials.

©2024 Percona

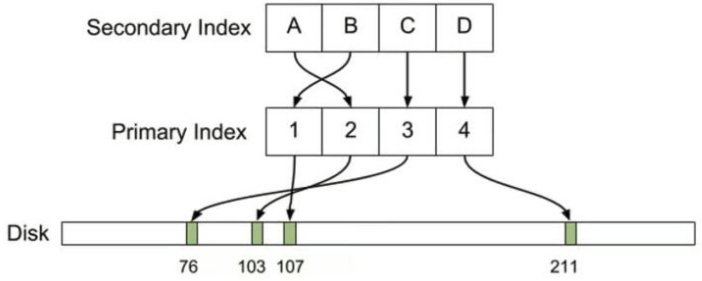
Write Amplification



Engineering

Why Uber Engineering Switched from Postgres to MySQL

26 July 2016 / Global



"a small logical update (say, writing a few bytes) becomes a much larger"

©2024 Percona

2. Write amplification in secondary index

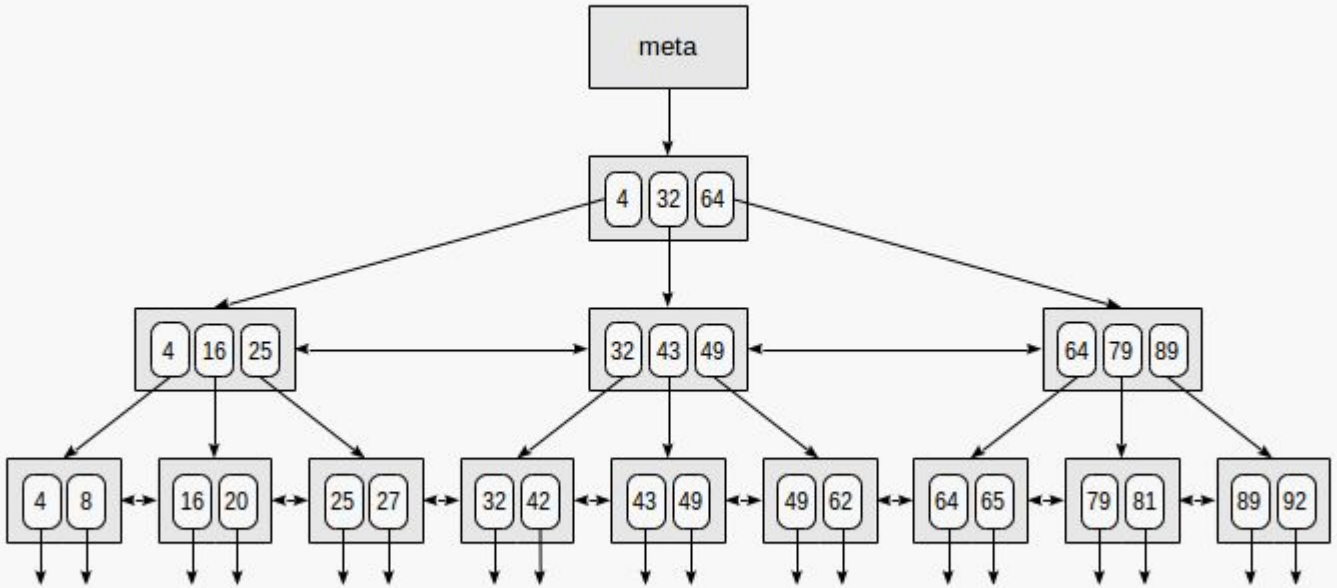
- UPDATEs causes new tuple in Table
- All indexes need to be updated pointing to new tuple

Logically, PostgreSQL need to update every index, irrespective whether the updated column is part of index.

©2024 Percona

3. Write Amplification due scattered storage

Btree



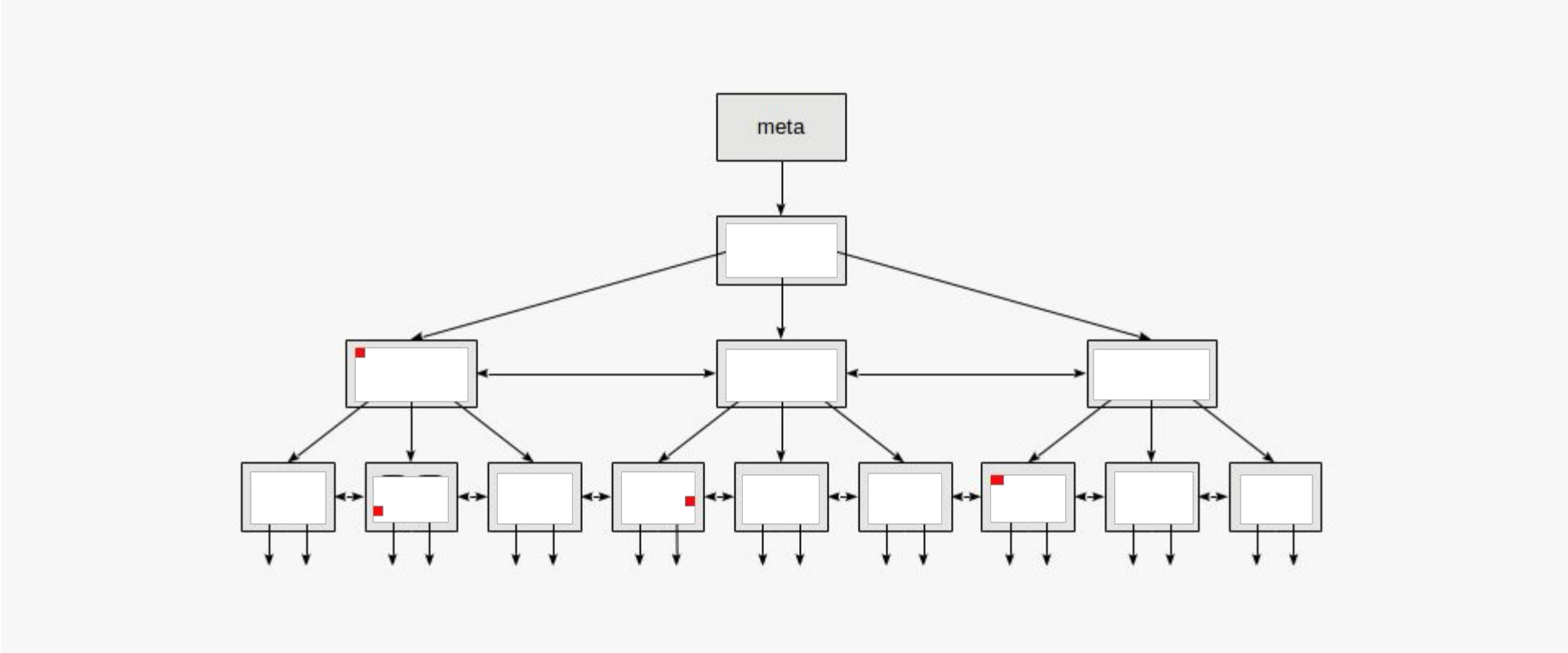
Even continuous data goes to different locations

Index access is Random Write

Egor Rogov, Indexes in PostgreSQL — 4 (Btree) https://postgrespro.com/blog/pgsql/4161516

©2024 Percona

4. Write amplification due to page size



©2024 Percona

5. Read Amplification due to Read-Ahead

- **PostgreSQL don't do Direct IO.**
- **Uses Linux Page Cache**
- **Read-Ahead:** The kernel employs a read-ahead mechanism to prefetch data into the page cache. When an application reads a certain amount of data, the kernel will typically read additional pages beyond the requested amount.
- **Random Read** Requests for updating Index

©2024 Percona

6. Needs more memory to cache

- Random read and write requires more memory for cache

©2024 Percona

Indications of poor cache

DB Server Time - Wait-events, CPU time and Delays (Reference)

Event	
ClientRead	13040
DataFileRead	2267
CPU	2216
ArchiveCommand	1999
WALSync	737
WALWrite	555
DataFileFlush	70
DataFileWrite	12

Clean by checkpoints (%)	Clean by bgwriter (%)	Clean by backends (%)
23.8	9.9	66.3

pg_gather

©2024 Percona

Misunderstanding about Unused Indexes

Indexes in [illegible] DB

Table	Index	UK?	PK?	Scans	size	Fetch	C.Hit%	Last Use
[illegible]	ix_[illegible]	f	f	53484430117	19947520	160453278926	99	
[illegible]	ix_[illegible]	f	f	**0**	422846464	**837962891**	99	
[illegible]	ix_[illegible]	f	f	**0**	263176192	**466994334**	99	
[illegible]	ix_[illegible]	f	f	89158700	48570368	267476385	99	
[illegible]	ix_[illegible]	f	f	**0**	93986816	**153661281**	99	
[illegible]	ix_[illegible]	f	f	**0**	93986816	**153445360**	99	
[illegible]	ix_[illegible]	f	f	21131766	40599552	63395327	99	

- Even Unused indexes need to be read into memory for update
- Unused Indexes also need to be updated by every DML
- Unused indexes also consumes memory

All expenses without any yield / benefit

©2024 Percona

7. Increased WAL generation

Database time

Wait Events and CPU info.

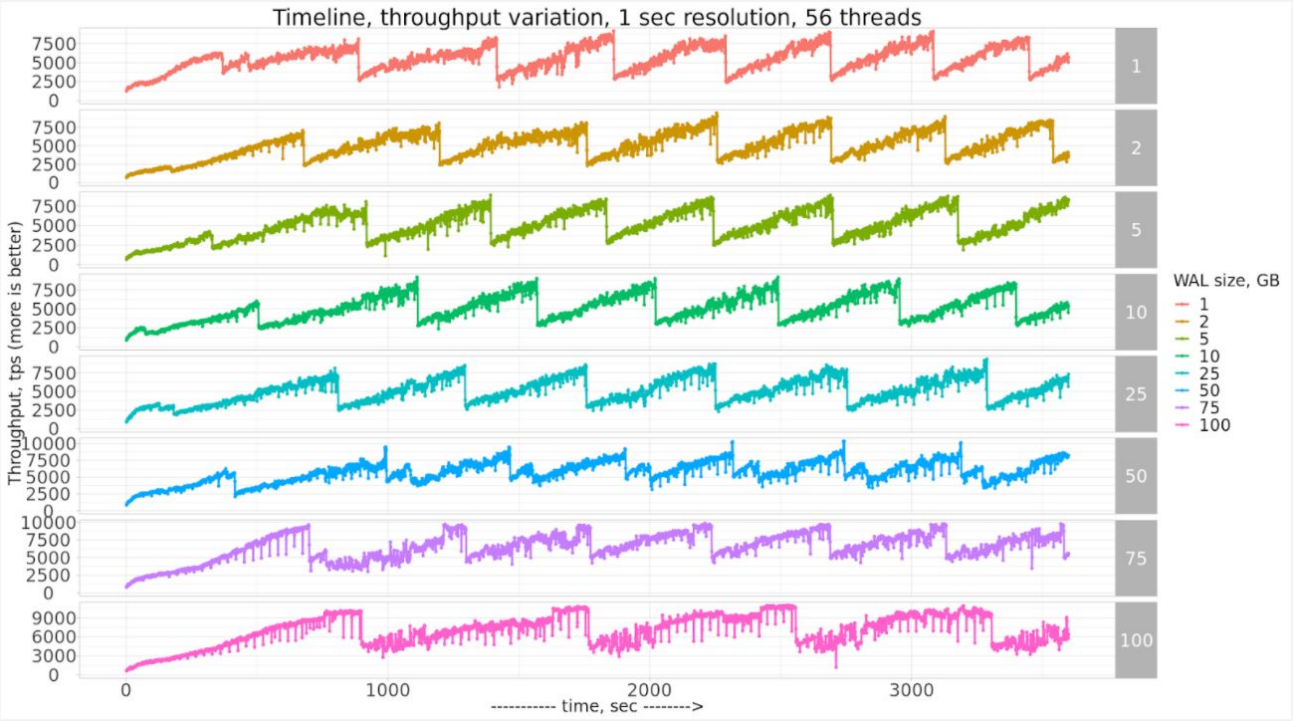
Event	count
WALWrite	4508
WALSync	1908
CPU	953
DataFileWrite	478
DataFileRead	404
ClientRead	231
transactionid	10
BufferContent	1
DataFileExtend	1

Wait Event Reference

- WAL generation rate has direct effect on TPS performance

©2024 Percona

8. Amplification of WAL Writes because of FPW




```
     fpi_rate
------------------------
 0.62856645571569103624
(1 row)
```

- Effect is multiplied due to **Full Page Writes.**
  - FPW depends on the number of pages dirited after checkpoint

©2024 Percona

Consequence Excessive WAL generation

- Replication Need more data to be transferred over network
- Replication apply has more work to do.
- Bigger backups, High Backup storage consumption
- WAL archiving not able to catch up with generation

©2024 Percona

9. Index bloating and Autovacuum

- Indexes also gets bloated
- Gradually indexes becomes inefficient.
- Indexes also need to be vacuumed as part of table vacuum.
- Generally takes more time and effort

- More processing
- More WAL generation and datafile writes
- More IO.

©2024 Percona

Indexes prevents HOT update

HOT update optimization is possible only when

- The update **does not modify any columns referenced by the table's indexes**, not including summarizing indexes.

©2024 Percona

Blogs, Studies & Cases

Insight for DBAs | Insight for Developers | PostgreSQL

Subscribe to RSS Feed

PostgreSQL Indexes Can Hurt Performance: Exploring the Negative Effects and the Costs Involved

April 1, 2024

Jobin Augustine

This blog was originally published in April 2023 and was updated in April 2024.

Indexes are generally considered to be the panacea when it comes to SQL performance tuning, and PostgreSQL supports different types of indexes catering to different use cases. I keep seeing many articles and talks on "tuning" discussing how creating new indexes speeds up SQL but rarely ones discussing removing them. The urge to create more and more indexes is found to be causing severe damage in many systems. Many times, removing indexes is what we should be doing first before considering any new indexes for the benefit of the entire system. Surprised? Understanding the consequences and overhead of indexes can help to make an informed decision and potentially save the system from many potential problems.

At a very basic level, we should remember that indexes are not free of cost. The benefits come with a

https://www.percona.com/blog/postgresql-indexes-can-hurt-you-negative-effects-and-the-costs-involved/

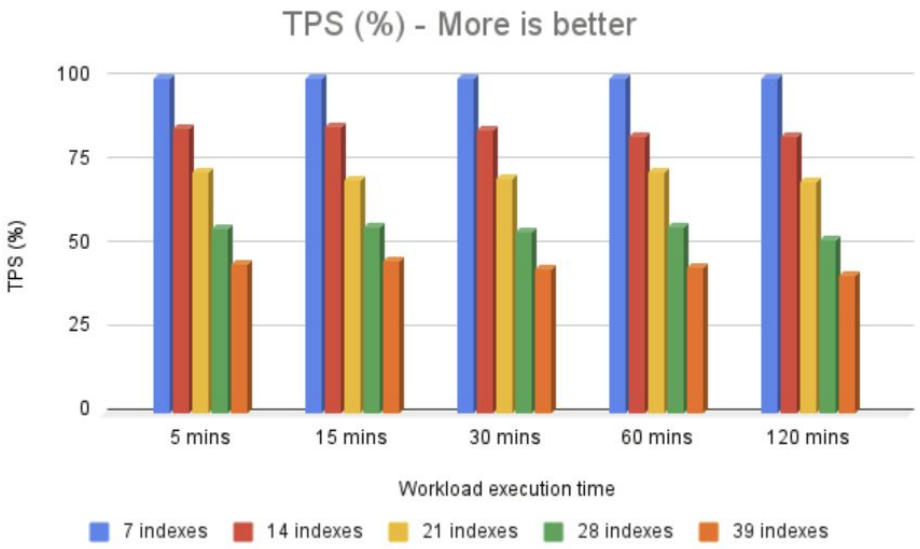
©2024 Percona

Benchr

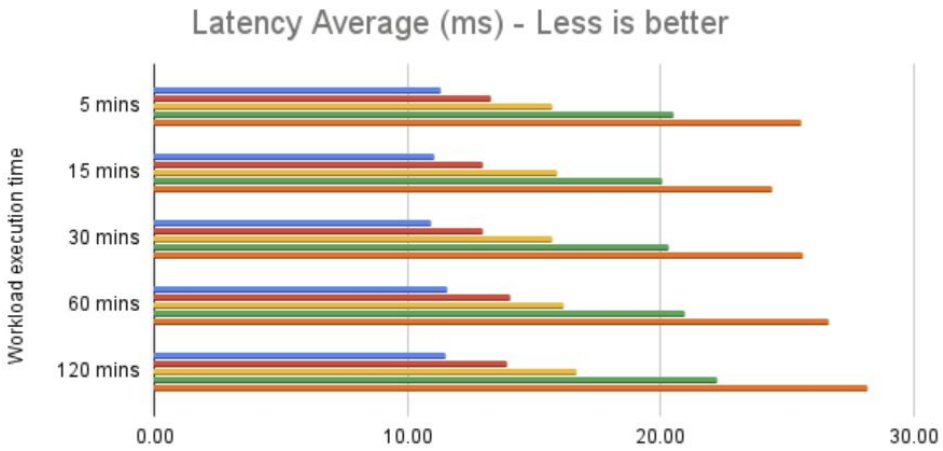
April 30, 2025

Indexing

Pablo Svampa



In other words, increasing the number of indexes from 7 to 39 reduced performance to approximately 42% of the original throughput. This clearly highlights the inverse relationship between index count and performance in our benchmark.



https://www.percona.com/blog/benchmarking-postgresql-the-hidden-cost-of-over-indexing/

©2024 Percona

Big Index, Big problem

MinervaDB | Products | Support | Remote DBA Services | ClickHouse | Blog | Careers | Education | Contact

FACEBOOK | TWITTER | LINKEDIN | GITHUB | YOUTUBE | HASHNODE | DEV | MEDIUM | POSTGRESQL BLOG

HOME > MINERVADB > How large Indexes corrupt PostgreSQL Execution Plans?

How large Indexes corrupt PostgreSQL Execution Plans?

MinervaDB, PostgreSQL, PostgreSQL DBA, PostgreSQL Internals, PostgreSQL Performance, PostgreSQL Troubleshooting

https://minervadb.xyz/how-large-indexes-corrupt-postgresql-execution-plans/

©2024 Percona

Indexes and Vacuum

boringSQL

Products Guides Talks Archive About

Home » Posts » VACUUM Is a Lie (About Your Indexes)

VACUUM Is a Lie (About Your Indexes)

2025-12-11 · 11 min · Radim Marek

TABLE OF CONTENTS

There is common misconception that troubles most developers using PostgreSQL: tune VACUUM or run VACUUM, and your database will stay healthy. Dead tuples will get cleaned up. Transaction IDs recycled. Space reclaimed. Your database will live happily ever after.

But there are couple of dirty "secrets" people are not aware of. First of them being **VACUUM is lying to you about your indexes**.

UPCOMING EVENTS

24 Mar 26
Nordic PgDay 2026, Helsinki

4-5 Jun 26
PG Data Conference 2026, Chicago

8 Sep 26
PgDay UK 2026, London

View past talks and slides →

https://boringsql.com/posts/vacuum-is-lie/

©2024 Percona

Planner & Statistics

Conserving CPU's cycles ...

500 Milliseconds on Planning: How PostgreSQL Statistics Slowed Down a Query 20 Times Over

When legacy decisions meet current database realities

ANDREI LEPIKHOV

JAN 28, 2026

6 1 Share

A query executes in just 2 milliseconds, yet its planning phase takes 500 ms. The database is reasonably sized, the query involves 9 tables, and the default_statistics_target is set to only 500. Where does this discrepancy come from?

This question was recently raised on the pgsql-performance mailing list, and the investigation revealed a somewhat surprising culprit: the column statistics stored in PostgreSQL's pg_statistic table.

https://danolivo.substack.com/p/500-milliseconds-on-planning-how

https://github.com/danolivo/conf/blob/main/2026-massive-table-stat/query-explain-1c.txt

©2024 Percona

Suggestions & Guidelines

Awareness

- Use Indexes only when there is no other way to meet the query performance expectations.
- Be aware about all negative side effects.
- Avoid copying Indexes from other systems. Because performance characteristics of PostgreSQL is different.
- Use tools / Scripts to identify effectiveness of indexes
  - Unused indexes are the first to be removed from the system
- Periodically review and evaluate the usage and performance

©2024 Percona

Use new versions of PostgreSQL

- Many Index optimizations are added in new features
- Now it is easy to find out when was last time an index is used

Index	UK?	PK?	Scans	size	Fetch	C.Hit%	Last Use
[illegible]	t	f	0	21131100160	1345417530	96	
[illegible]	t	f	0	21131042816	1345413322	96	
[illegible]	t	t	0	7943503872	697465129	99	
[illegible]	t	f	0	5367898112	283539118	99	
[illegible]	t	f	0	5367808000	283415807	99	
[illegible]	t	f	83530594	2573017088	500668707	99	2025-02-19 16:55:11

©2024 Percona

Thank You!

©2024 Percona

Title here

Poppins Medium size 18

- Poppins Normal size 18
- Poppins Normal size 18
- Poppins Normal size 18

Poppins Medium size 18

- Poppins Normal size 18
- Poppins Normal size 18
- Poppins Normal size 18

©2024 Percona

Title here - Poppins Medium size 34 colour #oe1a53

Poppins Normal size 20 - colour #oe1a53

©2024 Percona

Title Here

Text - Poppins Normal size 18

©2024 Percona

Title Here

Text - Poppins Normal size 18

©2024 Percona

Title Here

Text - Poppins Normal size 18

Text - Poppins Normal size 18

©2024 Percona

Section Cover

Subhead

Section Cover

Subhead

Section Cover

Subhead

Section Cover

Subhead

Section Cover

Subhead

Title Goes Here

Subtitle Goes Here

©2024 Percona

Extras

Poppins Bold Size 17

Poppins Normal 12

Poppins Bold Size 17

Poppins Normal 12

Poppins Bold Size 17

Poppins Normal 12

Poppins Bold Size 17

Poppins Normal 12

Poppins Bold 24

Poppins Normal 16

©2024 Percona

Title here -Poppins Medium Size 34

Subtitle - Poppins Semibold Size 18

Poppins Normal size 13

Poppins Normal size 13

Poppins Normal size 13

Poppins Normal size 13

Poppins Normal size 13

Poppins Normal size 13

©2024 Percona

Title

Subhead

Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13

- Poppins Normal size 13
- Poppins Normal size 13
- Poppins Normal size 13
- Poppins Normal size 13

Title

Subhead

Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13

- Poppins Normal size 13
- Poppins Normal size 13
- Poppins Normal size 13
- Poppins Normal size 13

Title

Subhead

Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13
Poppins Normal size 13

- Poppins Normal size 13
- Poppins Normal size 13
- Poppins Normal size 13
- Poppins Normal size 13

©2024 Percona

Title here

Subtitle

Poppins Normal size
13

Poppins Normal size
13

Poppins Normal size
13

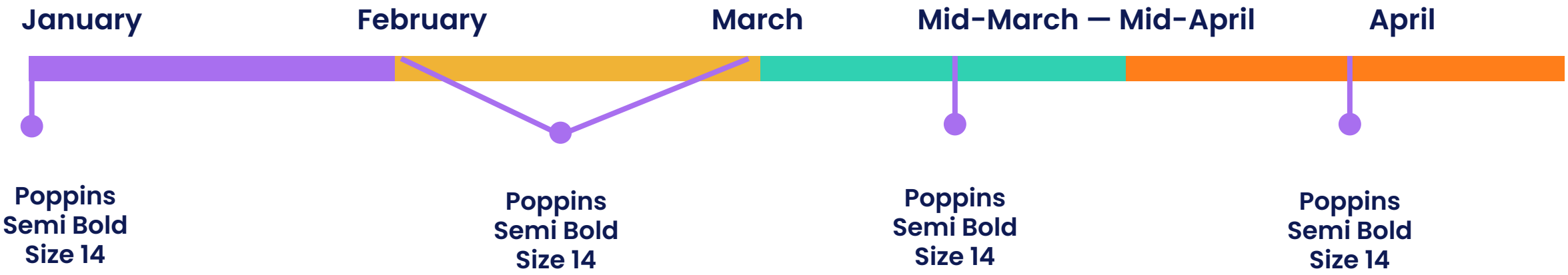
Poppins Normal size
13

Poppins Normal size
13

Poppins Normal size
13

©2024 Percona

Timeline



©2024 Percona

Title here

Text
Goes Here

Call out Text

Text
Goes Here

Call out Text

Text
Goes Here

Call out Text

Text
Goes Here

Call out Text

Text
Goes Here

Call out Text

©2024 Percona